What is CNN?

CNN stands for **Convolutional Neural Network**. It is a type of deep learning neural network that is especially effective for processing images.

Suppose we give a CNN an image and ask, **"What is present in this image?"** The model does not look at the image in the same way a human does. Instead, the image is represented as numerical values, and the CNN learns patterns from those values.

Face recognition, where the model learns patterns from faces.

Medical image analysis, where the model can help identify patterns in X-rays or scans.

Object detection, where the model identifies objects such as cars, people, or traffic signs.

The important thing to understand is that **CNN learns visual patterns from images.**

For example, if the image contains a cat, the CNN may gradually learn simple features such as edges and curves and then combine them into more meaningful features such as ears, eyes, whiskers and shapes. Eventually, these features can be used to predict that the image contains a cat.

How Does a Computer See an Image?

When we look at an image, we can easily understand what is present in it. For example, if we see a picture of a cat, our brain immediately recognizes the cat.

But a computer does not see the image in the same way we do.

A digital image is represented using **numbers**. These numbers represent the pixels of the image.

Before understanding CNN, we need to understand pixels.

A digital image is made up of many small elements called **pixels**, which means picture elements.

Each pixel contains numerical information representing the intensity or color at that location.

Imagine we have two photographs of Abraham Lincoln.

One is a **color image**, and the other is a **grayscale image**.

As humans, we can easily understand both images. We can recognize that they are photographs of the same person even though one contains color information and the other mainly contains brightness information.

But the CNN does not receive the image as the concept **“Abraham Lincoln.”**

It receives numerical information representing the image.

This is where pixels become important.

A pixel is simply a small unit of an image.

For a grayscale image, a commonly used representation is a value from **0 to 255**. A value close to 0 represents black, while a value close to 255 represents white. Values between them represent different shades of gray.

For example:

0 → Black

50 → Dark gray

128 → Gray

200 → Light gray

255 → White

Therefore, a grayscale image can be represented as a matrix of numbers.

For example, a very small portion of an image could look conceptually like:

0 20 50

100 150 200

220 240 255

These numbers represent the brightness of the pixels.

What About a Color Image?

A grayscale image normally has only one channel because it contains information about brightness.

For example, if we have an image of size **28 × 28**, a grayscale image can be represented as:

28 × 28 × 1

Here, 28 represents the height, 28 represents the width, and 1 represents the single grayscale channel.

A color image usually contains three channels:

Red, Green and Blue (RGB).

Therefore, a color image does not normally have just one number for each pixel. It has three values associated with each pixel, one for each color channel.

a color image of the same spatial size could be represented as:

28 × 28 × 3

Here, the three channels represent Red, Green, and Blue.

A CNN can process color images directly using their multiple channels.

How Does CNN Recognize an Image?

Imagine that we give a CNN an image containing the handwritten number **3**.

A human can immediately recognize it as 3 because we understand its overall shape. We do not consciously examine every individual pixel.

CNN works differently. It processes the numerical image and gradually learns useful visual patterns. At the beginning, it may detect simple features such as edges and curves. In deeper layers, these simple features can be combined into more complex shapes. Eventually, the network can use these learned features to determine that the image represents the number 3.

This is why CNN is powerful for image recognition.

7. Convolution Layer

Now imagine that we give a complete image to the neural network.

The image may contain thousands or millions of pixels. If we simply connect every pixel to every neuron, the number of parameters can become extremely large.

CNN solves this problem by looking at small regions of the image at a time.

This is where the important concept called convolution comes into the picture.

The first major operation in a CNN is the **convolution operation**, which is performed using filters or kernels.

A **kernel**, also called a **filter**, is a small matrix of numbers that moves across different regions of the image. Common kernel sizes include **3 × 3** and **5 × 5**.

For example, if we use a 3 × 3 kernel, the CNN examines a small 3 × 3 region of the image at a time. The kernel performs mathematical calculations with the pixel values in that region and then moves to another region.

This process allows the CNN to detect useful patterns in different parts of the image.

What Does the Convolution Layer Learn?

The convolution layer learns features from the image. In the early layers, the network commonly learns relatively simple features such as **edges, lines and basic curves**.

The first convolution layers generally learn **simple visual patterns**.

For example, they may learn to respond to:

Edges

Lines

Corners

Simple curves

Different textures

Suppose we give a picture of a cat to the CNN.

The first layer does not necessarily understand that it is a cat. Instead, it may detect simple patterns such as edges and curves.

For example, one filter may respond strongly to a vertical edge, while another may respond to a horizontal edge.

So the first layers are mainly learning **simple features**.

As the network becomes deeper, the learned features become more complex. Later layers can combine simple edges and curves to recognize shapes and object parts.

Therefore, the learning process can be understood conceptually as:

Simple features → combinations of features → complex features → object recognition

The exact features learned are not manually programmed by us. The network learns suitable filters during training.

Understanding the 3 × 3 Example

Suppose our image contains many pixels.

Instead of processing the entire image as one giant block, CNN can use a small **3 × 3 kernel**.

The kernel looks at nine values at a time.

For example:

1 2 3

4 5 6

7 8 9

The filter performs a mathematical operation using this local region.

Then the filter moves to another nearby region.

It might move one step to the right and perform the calculation again.

It continues moving across the image until the required regions have been processed.

The filter can also be **5 × 5** or another suitable size depending on the architecture.

Detective Example for Understanding Filters

A good way to understand filters is to imagine that we have a group of **detectives**.

Suppose there is a large image and we want to find different clues inside that image.

Each detective has a different job.

One detective might look for **horizontal lines**.

Another detective might look for **vertical lines**.

Another might look for **edges**.

Another might look for **curves**.

The filter behaves somewhat like these detectives. Each filter looks at the image and tries to detect a particular type of visual pattern.

So we can think of:

Image → Case

Filter → Detective

Pattern detected by filter → Clue

This is only an analogy, but it helps us understand the basic idea.

What is a Feature Map?

When a filter moves across an image and performs convolution, it produces a new numerical representation called a **feature map**. .

A feature map tells us where a particular learned pattern is present in the image.

For example, suppose a filter is designed to respond strongly to a certain type of edge.

When that filter moves across the image, the locations where that edge appears will produce stronger values in the feature map.

So we can think of it as:

Input image + filter/kernel → convolution → feature map

Multiple filters can produce multiple feature maps, allowing the network to capture different types of features.

Why Are There Multiple Convolution Layers?

A CNN usually contains multiple convolution layers because different layers can learn features at different levels of complexity.

The earlier convolution layers generally learn simpler patterns such as edges, lines and curves. Later convolution layers can combine these simpler patterns to learn more complex patterns.

For example, when recognizing a cat, an early layer may detect edges. A later layer may combine edges into shapes. A deeper layer may recognize parts such as ears or eyes. Eventually, the network can combine these features and identify the overall object as a cat.

For handwritten digit recognition, early layers might detect curves and edges, while deeper layers can combine these patterns to recognize that the overall shape corresponds to the number 3.

This is called **hierarchical feature learning**.

---

14. What is Pooling?

After convolution, CNNs commonly use another operation called **pooling**.

The main purpose of pooling is to reduce the spatial size of the feature maps while keeping important information.

Think about taking a large image and creating a smaller version that still preserves the important visual information. Pooling performs a similar type of reduction on feature maps.

One common type is **Max Pooling**.

In max pooling, we look at a small region and select the largest value.

For example, suppose we have this 2 × 2 region:

2 8

3 1

The largest value is **8**.

Therefore, max pooling keeps **8** and removes the other values from that local region.

The idea is that a strong activation may indicate the presence of an important feature, so max pooling retains that strong response while reducing the amount of information that needs to be processed. .

---

15. Why Do We Use Pooling?

Pooling helps reduce the spatial dimensions of the data. This means the network has fewer values to process in later stages.

It can also make the representation somewhat less sensitive to small changes in the exact location of a feature. For example, if an edge shifts slightly within an image, the pooled representation may still retain a strong response.

Therefore, pooling helps make the network more computationally efficient while retaining important information.

Convolution and Pooling Together

CNN usually repeats the process of convolution and pooling.

For example:

Input Image → Convolution → Feature Map → Pooling → Convolution → Feature Map → Pooling

The first convolution layers learn simple patterns.

As we move deeper into the network, the model can combine those patterns into more complex features.

Pooling helps reduce the spatial size along the way.

Therefore, the network gradually transforms the original image into a more compact representation containing useful information for classification.

Deeper Convolution Layers

As we move deeper into the CNN, the network works with increasingly meaningful representations.

The early layers may detect simple patterns such as edges and curves. The middle layers can combine these patterns into shapes or object parts. The deeper layers can combine those parts into higher-level concepts.

For example, in a cat image, the progression can be understood conceptually as:

Edges → curves → shapes → eyes/ears/whiskers → cat-like representation

Fully Connected Layer

After the convolution and pooling stages have extracted useful features, the information can be passed to a fully connected layer, also called a dense layer.

The extracted features are passed into the classification part of the network.

Before entering the fully connected layer, the feature maps are typically transformed into a one-dimensional representation, commonly using a flattening operation or another suitable feature aggregation method.

The fully connected layers use the learned features to help make the final prediction.

Let's take a simple example.

Suppose we want to build a CNN that can recognize handwritten digits from **0 to 9**.

We give the model an image of a handwritten digit.

Suppose the image is:

28 × 28 × 1

because it is a 28 × 28 grayscale image.

The image enters the CNN.

The convolution layers examine different regions of the image and learn patterns such as edges, curves, and shapes.

Pooling reduces the spatial dimensions.

Deeper convolution layers combine simpler patterns into more complex shapes.

Eventually, the extracted information is passed to the Dense layers.

Finally, the output layer gives probabilities for all ten possible classes.

---

16. Output Layer and Probability

The final output layer produces the prediction.

Suppose we are building a CNN to recognize handwritten digits from **0 to 9**. The output layer can contain 10 output values, one corresponding to each digit.

The model may produce probabilities such as:

0 → 2%

1 → 3%

2 → 1%

3 → 88%

4 → 1%

5 → 2%

6 → 1%

7 → 1%

8 → 0.5%

9 → 0.5%

In this example, the highest probability is for **3**, so the model predicts that the image represents the digit 3.

The important point is that the model's prediction is based on learned numerical patterns and probabilities. It is not necessarily "100% certain."

.

Complete CNN Process

Now we can understand the complete CNN process.

First, we give the image to the network.

The image is represented numerically as pixels.

The convolution layers apply filters to small regions of the image and learn useful patterns.

The first layers generally learn simple patterns such as edges and lines.

Deeper layers combine these patterns into more complex visual features.

Pooling reduces the spatial dimensions while retaining important information.

After several convolution and pooling operations, the learned features are passed to the Dense or fully connected layers.

The final output layer produces probabilities for the possible classes.

The class with the highest probability is generally selected as the prediction.

Therefore, the overall flow is:

Image → Pixels → Convolution → Feature Maps → Pooling → Deeper Feature Learning → Dense Layer → Output Probabilities → Prediction

First, the user provides an image. The image is represented numerically using pixels and, for color images, multiple channels. The image is then passed through convolution operations where filters learn and detect useful patterns. These operations produce feature maps. Pooling may then reduce the spatial dimensions while retaining important information. Multiple convolution and pooling stages can progressively learn more complex features. The resulting features are then passed to fully connected or other classification layers, and finally the output layer produces probabilities for the possible classes.